BATCH 26

Inceptez Data Science Presents

Python Day - 4

Laxmi Narayen Nagarajan Venkatesan

26th-April-2026

PART 1

Optimizing Looping Strategies

For Loops: Definite Iteration

Ideal for iterating over known sequences like `lists`, `strings`, or `ranges`. Use when the number of steps is pre-determined.

While Loops: Indefinite Iteration

Used when the number of iterations is unknown. Executes as long as a condition remains `True`. Perfect for event listening or state-based logic.

💡 **Pro Tip:** Choosing the right tool for the job ensures clean, readable, and performant code.

PART 2

Entry Control: While & For Loops

The While Loop (Indefinite)

Executes as long as the condition is `True`. The condition is checked **before** every iteration.

Code Pattern:

```
count = 0
while count < 5:
  print(count)
  count += 1
```

Ideal for: User input validation, game loops, or reading unknown data streams.

The For Loop (Definite)

Iterates over a sequence (list, tuple, string). It is also an entry-controlled loop.

Code Pattern:

```
fruits = ["apple", "banana"]
for fruit in fruits:
print(fruit)
```

Ideal for: Iterating over fixed collections or numerical ranges where steps are known.

💡 **Entry Control:** In Python, both For and While loops check conditions at the start, ensuring code only runs if valid.

PART 3

Pythonic Elegance: List Comprehensions

The "Why": Optimized Iteration

- Reduces syntactic noise by replacing multi-line loops.
- Faster execution due to internal C-level optimization.
- Combines mapping and filtering into a single expression.

The Syntax Pattern

```
[expression for item in iterable if condition]
```

A concise way to create lists from existing sequences while applying logic on the fly.

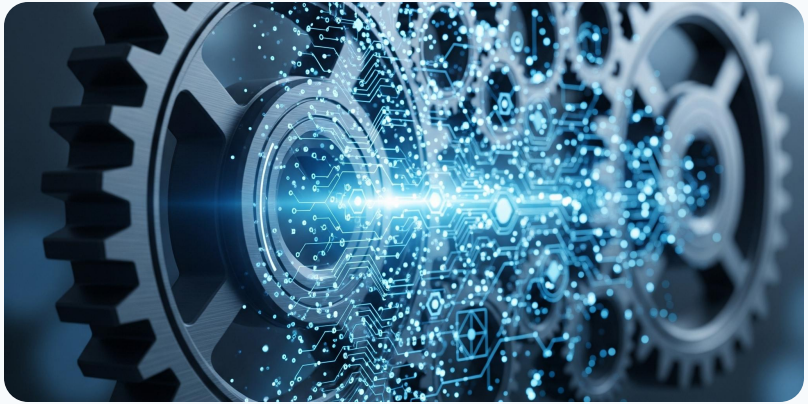
💡 **Pythonic Rule:** Use comprehensions for clarity. If the logic is too complex for one line, stick to a standard For Loop.

PART 4

Modularity & Reusability: Python Functions

The "Why": Structural Optimization

- Eliminates redundancy by adhering to the DRY principle (Don't Repeat Yourself).
- Improves maintainability: update logic in one place, reflect everywhere.
- Enhances readability by abstracting complex operations into named blocks.



Defining Logic with `def`

```
def calculate_ai_metric(data, threshold=0.5):
 return [x for x in data if x > threshold]
```

Functions allow for flexible data processing using parameters and default values.

💡 **Scalability Tip:** Functions are the building blocks of production-ready AI pipelines, enabling modular testing and debugging.

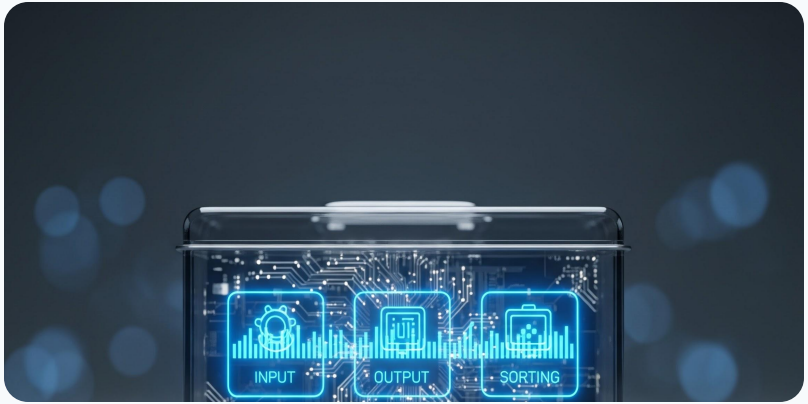
PART 5

Built-in Power: Python's Default Functions

Essential Utility Functions

Python provides a rich set of pre-defined tools to handle common tasks without extra imports.

- `print()` & `input()`: I/O operations
- `len()`: Collection sizing
- `type()`: Dynamic inspection



Numeric & Sequence Helpers

Optimized functions for mathematical operations and sequence transformations.

- `sum()`, `min()`, `max()`: Statistics
- `range()`: Iteration control
- `sorted()`: Non-destructive sorting

💡 **Battery Included:** Leverage built-ins for performance; they are implemented in C and are faster than custom logic.

EXAMPLES

Mastering the Print Statement

The `sep` Parameter

Controls the character inserted between multiple values. Default is a space.

```
print("A", "B", "C", sep="-")
# Output: A-B-C
```

The `end` Parameter

Specifies what to print at the very end. Default is a newline (\n).

```
print("Hello", end="! ")
print("World")
# Output: Hello! World
```

ADVANCED

Flexible Interfaces: Function Parameters

Default Parameters

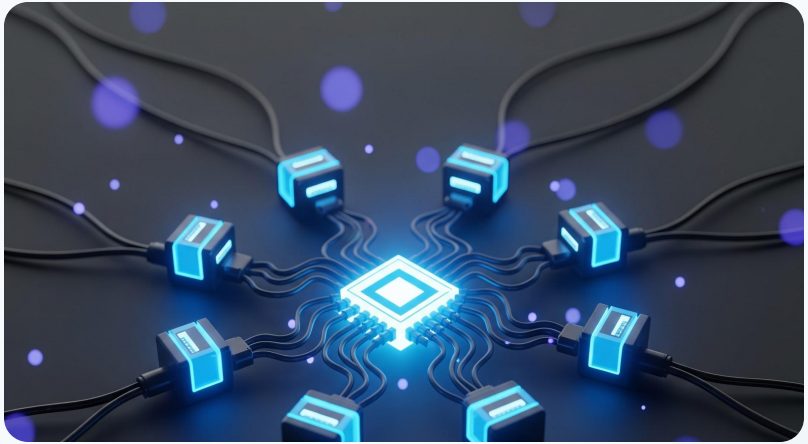
Assign values in the definition to make arguments optional.

```
def greet(name, msg="Hello"):
  print(f"{msg}, {name}")
```

Keyword Arguments

Call functions using name=value to ignore positional order.

```
greet(msg="Hi", name="Alice")
# Output: Hi, Alice
```



💡 **Best Practice:** Place default parameters after non-default ones to avoid syntax errors. Default values are evaluated once at definition.

CHALLENGE

Function Practice: The Max Finder

Coding Exercise

Create a function to find the maximum of four numbers. This solidifies your understanding of basic syntax and logical flow.

Implementation Detail

```
def max_of_four(a, b, c, d):
  return max(a, b, c, d)
```

Measure your knowledge and have fun coding with these interactive exercises.

Learning Tip

Practice is key. Interactive challenges help visualize data flow and execution steps.

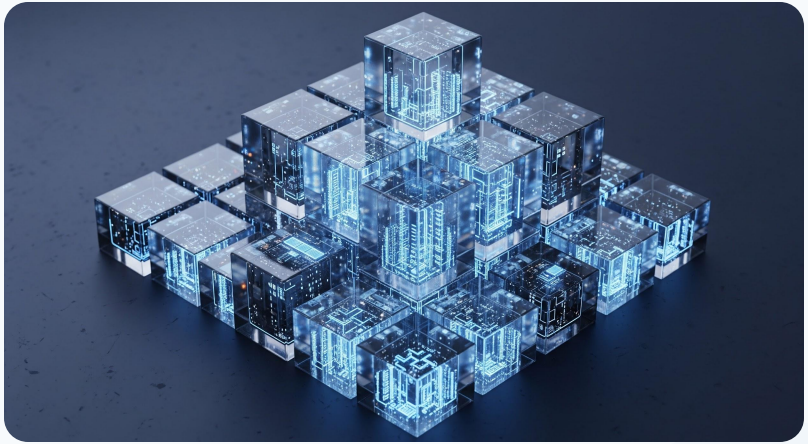
💡 **Pro Tip:** Regular practice with functions builds the foundation for complex AI application development.

PART 6

Object-Oriented Programming (OOP)

Core Pillars of OOP

- **Encapsulation:** Bundling data and methods.
- **Inheritance:** Creating new classes from existing ones.
- **Polymorphism:** Using a single interface for different types.
- **Abstraction:** Hiding complex implementation details.



Class vs. Instance

```
class AIModel:
  def __init__(self, name):
    self.name = name
```

A **Class** is the blueprint; an **Instance** is the actual object created from that blueprint.

Real-World Analogy

Think of a "Car" blueprint. Every specific car on the road is an instance of that blueprint, with its own color and speed.

💡 **OOP Advantage:** Organizing code into objects makes large-scale AI applications easier to maintain and extend.

EXAMPLES

OOP Pillars: Practical Code Examples

Encapsulation & Abstraction

```
class BankAccount:
  def __init__(self):
    self.__balance = 0 # Private

  def deposit(self, amount):
    self.__balance += amount
```

Encapsulation bundles balance and methods. **Abstraction** hides the internal math logic from the user.

Inheritance

```
class Animal:
  def speak(self): pass

class Dog(Animal): # Inherits Animal
  def speak(self):
    return "Woof!"
```

The **Dog** class reuses properties from the **Animal** base class, demonstrating code reusability.

Polymorphism

```
def make_it_speak(animal):
  print(animal.speak())

my_dog = Dog()
make_it_speak(my_dog) # "Woof!"
```

Polymorphism allows different classes (like Dog or Cat) to be treated through the same interface.

DEEP DIVE

Types of Inheritance in Python

Single & Multiple Inheritance

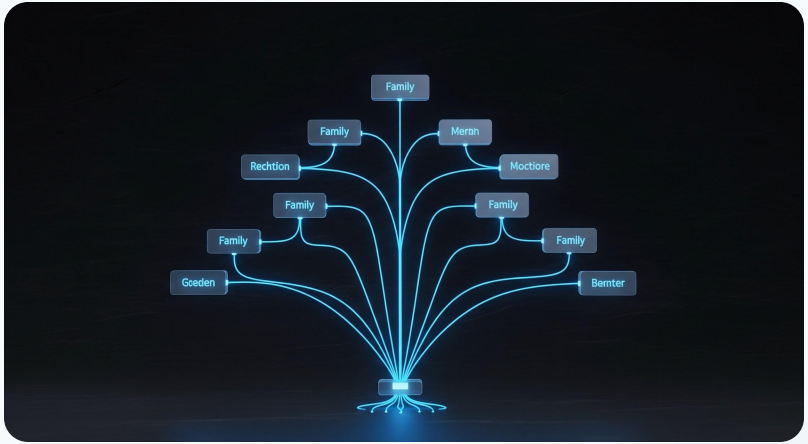
Single: A child class inherits from one parent.
Multiple: A child class inherits from more than one base class.

```
class Child(Parent1, Parent2):
```

Multilevel & Hierarchical

Multilevel: A chain of inheritance (Parent → Child → Grandchild).
Hierarchical: Multiple classes inherit from one parent.

```
class Puppy(Dog): # Multilevel
```



Hybrid Inheritance

A combination of multiple and multi-level inheritance structures within one program.

💡 **Key Advantage:** Inheritance facilitates code reusability and improved readability in large systems.